

Incident Analysis Report: Windows Event Logs

Investigation Summary

This report summarizes the findings from the Windows Event Log analysis performed on the target system (ACADEMY-SFUND-WIN10).

Question 1: Modification of Auditing Settings

- **Task:** Identify the executable responsible for the audit settings modification.
- **Finding:** By analyzing the security logs and correlating events with the specific timestamp, the process was identified.
- **Answer:** `TiWorker.exe`

Question 2: Audit Modification Time

- **Task:** Determine the exact time when the auditing settings for the specified file were modified.
 - **Target File:**
`C:\Windows\Microsoft.NET\Framework64\v4.0.30319\WPF\wpfgfx_v0400.dll`
 - **Methodology:** Used a custom XML query in the Event Viewer to filter Event ID 4907 (Audit Policy Change) specifically for the target object.
 - **Finding:** The logs indicated an audit success event for the object.
 - **Answer:** `10:23:50`
-

Technical Notes for Future Reference

1. **Event ID 4624:** Used to trace successful logon sessions.
2. **Event ID 4907:** Crucial for detecting changes in System Access Control Lists (SACL), which often indicates an attacker attempting to hide their tracks or modify file auditing.
3. **XML Filtering:** A powerful technique for granular log analysis, especially when searching for specific file paths or object names.

Windows Event Logs & Finding Evil - Lab Summary

This document provides a concise summary of the exercises performed to analyze malicious activities using Sysmon and Event Logs.

1. DLL Hijacking

Objective: Detect unauthorized DLL loading by a legitimate application.

- **How it works:** Attackers rename a malicious DLL to match a legitimate one (e.g., `WININET.dll`) and place it in the same directory as a vulnerable executable (e.g., `calc.exe`). When the executable runs, it loads the malicious DLL instead of the real one from `System32`.
 - **Detection Strategy:**
 - **Tool:** Sysmon.
 - **Event ID:** 7 (Module Load).
 - **Indicators (IOCs):**
 - `calc.exe` running from a non-System32 directory.
 - `WININET.dll` being loaded by `calc.exe` from an unusual location.
 - The malicious DLL is unsigned.
-

2. Unmanaged PowerShell/C# Injection

Objective: Detect when C# code is injected into a process, turning it into a "managed" process.

- **How it works:** Managed languages like C# rely on the Common Language Runtime (CLR). If an attacker injects C# code into an unmanaged process (like `spoolsv.exe`), the process suddenly requires the CLR to function.
 - **Detection Strategy:**
 - **Tool:** Process Hacker / Sysmon.
 - **Event ID:** 7 (Module Load).
 - **Indicators (IOCs):**
 - Presence of `clr.dll` and `clrjit.dll` in processes that do not typically require them.
 - Process Hacker identifying a process as "Managed (.NET)".
-

3. Credential Dumping

Objective: Detect unauthorized attempts to extract password hashes from memory.

- **How it works:** Attackers use tools like Mimikatz to access the Local Security Authority Subsystem Service (LSASS), which stores user credentials. The command

`sekurlsa::logonpasswords` extracts these credentials.

- **Detection Strategy:**
 - **Tool:** Sysmon.
 - **Event ID:** 10 (Process Access).
 - **Indicators (IOCs):**
 - Unauthorized processes attempting to access `lsass.exe`.
 - A user requesting `SeDebugPrivilege` to perform the operation.
 - Mismatched "SourceUser" vs "TargetUser" when accessing LSASS.
-

Important Note

Always ensure that Sysmon is active and using an appropriate XML configuration file (like `sysmon-config` from GitHub) to ensure these events are captured accurately in the Windows Event Viewer.

Event Tracing for Windows (ETW) - Summary

Event Tracing for Windows (ETW) is a powerful, high-speed tracing facility built into the Windows kernel. It provides deep visibility into system activity, often capturing telemetry that traditional Event Logs (like Sysmon or Security logs) might miss.

1. ETW Architecture Components

ETW works on a **Publish-Subscribe** model:

- **Controllers:** Manage trace sessions (start/stop/enable/disable). The primary tool is `logman.exe`.
 - **Providers:** The sources that generate events (e.g., Kernel, Applications, Drivers).
 - **Consumers:** The applications that subscribe to events for processing or logging.
 - **Channels:** Logical containers used to organize and route events.
 - **ETL Files:** Event Trace Log files used for storing captured events for later analysis.
-

2. Key Tools for Interaction

- `logman.exe` : The main command-line utility for managing ETW sessions.
 - `logman.exe query -ets` : Lists all running trace sessions.

- `logman.exe query providers` : Lists all available event providers on the system.
 - `logman.exe query "SessionName" -ets` : Displays specific details of a session.
 - **Performance Monitor (GUI)**: A visual way to monitor and manage trace sessions.
 - **EtwExplorer**: A third-party utility for viewing detailed ETW provider metadata.
-

3. Why ETW Matters for Security

ETW is critical for Incident Response because it captures:

- **Kernel-level activity**: Registry modifications, file operations, and network connections.
 - **Low-level visibility**: Events from .NET runtime, PowerShell execution, and DNS requests.
 - **Efficiency**: It is lightweight and has minimal impact on system performance, making it ideal for real-time monitoring.
-

4. Notable Providers

Provider	Use Case
Microsoft-Windows-Kernel-Process	Detecting process injection and hollowing.
Microsoft-Windows-PowerShell	Tracking script block logging and command execution.
Microsoft-Windows-DNS-Client	Detecting DNS tunneling or C2 communication.
Microsoft-Windows-Kernel-Registry	Monitoring persistence and system config changes.
Microsoft-Windows-Threat-Intelligence	Restricted provider ; provides high-granularity data on sophisticated threats.

5. Important Note on Restricted Providers

Some high-value providers, such as `Microsoft-Windows-Threat-Intelligence`, are **restricted**. They require the process to run with **PPL (Protected Process Light)** privileges. This ensures that sensitive telemetry data is protected from tampering by malware.

Summary: Detecting Malicious .NET Activity with ETW

The Goal: Attackers often run malicious tools (like **Seatbelt**) directly in memory to avoid leaving files on the disk. This is hard to catch with standard security logs.

How we caught it: We used **ETW (Event Tracing for Windows)**, which is like a "deep recorder" for the Windows operating system. It allows us to see exactly what a program is doing inside the .NET runtime.

The Key Finding: When you ran Seatbelt, it needed to check your user permissions. To do this, it called a specific Windows function called **GetTokenInformation**.

- **Why it matters:** * **GetTokenInformation** starts with "**G**" and ends with "**ion**".
 - By monitoring these "Interop" calls (where managed .NET code talks to the Windows system), security teams can spot tools running in memory even if they look "invisible" to other tools.

In short: You used SilkETW to record the internal activity of the .NET framework, searched the logs, and found that the tool (Seatbelt) was calling **GetTokenInformation** to gather info about your account. This is a classic "signature" for this type of tool!

Analysis Note: Identifying Network Share Creation

1. Overview

In cybersecurity, attackers often create unauthorized network shares to move laterally across a network (Lateral Movement). We investigated logs to find exactly when the `\\*\PRINT` share was created.

2. Key Concept: Event ID 5142

In Windows security auditing, **Event ID 5142** is the specific event generated when a new network share object is added to the system. By monitoring this, we can track unauthorized resource creation.

3. The Investigation Process

To find the exact time, we performed the following:

- **Target:** Scanned the `Lateral Movement` folder containing multiple `.evtx` log files.
- **Tool:** Used the `Get-WinEvent` PowerShell cmdlet.
- **Filter:** Applied an `XPath` filter to look specifically for:
 - **Event ID:** 5142
 - **Data:** `ShareName` equal to `\\*\PRINT`

4. Final Result

The log file analysis confirmed that the event occurred at:

- **Result Time:** 12:30:30
-

Summary: By using PowerShell to filter Windows Security logs, we efficiently identified the exact moment the malicious/unauthorized share was created, which is a critical step in incident response.

Skills Assessment: Investigating Attack Logs

1. DLL Hijacking Investigation (C:\Logs\DLLHijack)

- **Objective:** Find the process that executed a DLL hijacking attack.
- **Method:** Check for Event ID 7 (Image Loaded) to see which process loaded a DLL from an unusual location.
- **Command Hint:** `Get-WinEvent -Path "C:\Logs\DLLHijack\*.evtx" | Select-Object -ExpandProperty Message | Select-String "Image"`.

2. Unmanaged PowerShell (C:\Logs\PowershellExec)

- **Objective:** Identify the process that ran unmanaged PowerShell code.
- **Method:** Use Event ID 1 (Process Creation) to find processes acting as a shell but are not `powershell.exe`.

3. Process Injection (C:\Logs\PowershellExec)

- **Objective:** Find the process that injected code into the PowerShell executor.
- **Method:** Filter for Event ID 8 (CreateRemoteThread) or ID 10 (ProcessAccess). Look at the `SourceImage` and `TargetImage` fields.

4. LSASS Dump (C:\Logs\Dump)

- **Objective:** Identify the process that dumped LSASS memory.
- **Method:** Search for Event ID 10 where the `TargetImage` is `lsass.exe` and the access rights are suspicious (e.g., `0x1F0FFF`).

5. Post-Dump Login (C:\Logs\Dump)

- **Objective:** Check if an unauthorized login occurred after the LSASS dump.
- **Method:** Find the timestamp of the LSASS dump (from step 4), then look for Event ID 4624 (Successful Logon) immediately following that time. Answer: **Yes** or **No**.

6. Strange Parent-Child Process (C:\Logs\StrangePPID)

- **Objective:** Find a process spawned by a "strange" parent.
 - **Method:** Check Event ID 1 (Process Creation) logs and compare the ParentImage with the Image . If the relationship looks abnormal (e.g., a system service spawning a shell), that is your answer.
-

*Tip: Always use `Select-Object -Property` to see all available data fields when a log entry looks interesting!**